Sri Lanka Institute of Information Technology

IT2130 – Operating Systems and System Administration
Year 2, Semester 2- 2026

Practical Answer Submission

Practical Sheet No: Lab_2

Student ID	IT24102099
Student Name	Mathuppriya Naguleswaran
Campus/ Center name	SLIIT – Northern UNI
Specialization	Software Engineering
Batch No	1

Selection statements in a C program

- **Selection statement :**It checks a condition and chooses which block of code should run.
  - It helps the program **choose one action** from many possible actions.
  - In C, common selection statements are if, if-else, and switch-case.

- A **condition** is a comparison used in a program to test whether something is true or false.

- The if statement is used to execute a block of code only when a given condition is true.

- The if else statement is used when there are **two choices**.
  - If the condition is true, the if block runs.
  - Otherwise, the else block runs.
  - if-else is used when decisions depend on conditions or ranges.

- else if is used when a program needs to check several conditions one after another and choose one matching result.

- A switch statement is used to select one block of code from multiple fixed options
  - Instead of many if statements, switch-case is cleaner.
  - break statement is used to stop the execution of the switch block after the correct case is executed.
  - Without break, the program may continue running the next cases as well.

- **Simple difference**
  - if else = good for conditions and ranges (using >, <, >=, <=)
  - switch = good for exact choices

- Input validation is important because it prevents invalid data from being processed.

- relational operators: >, <, >=, <=, ==

- logical operators: &&, ||

scanf(" %c", &gender) **with a space**

When reading a character, sometimes the Enter key from the previous input is still there. (it may read the newline instead of the actual character.) The space tells C to ignore extra blank spaces and newline.

A space is placed before %c in scanf(" %c", &gender) to ignore unwanted whitespace characters such as Enter or spaces.

- **A program can make decisions.**
- **To make decisions, it uses conditions.**
- **Conditions are true or false.**
- **if is used for one condition.**
- **if else is used for two choices.**
- **else if is used for many choices.**
- **switch is used for fixed options.**
- **break stops a switch case.**
- **default handles invalid choices.**
- **Variables store data.**
- **int, float, and char store different types of data.**
- **scanf() takes input.**
- **printf() shows output.**

logical operators : Logical operators are used to combine two or more conditions.
Why is input validation needed : Input validation is needed to make sure the entered value is correct and within the allowed range.

Quick check

- ✓ **If :** It checks a condition and runs code if true.
- ✓ **if else :** It chooses between two blocks.
- ✓ **else if :** It checks multiple conditions one by one.
- ✓ **Switch :** It selects one option from fixed values.
- ✓ **Break :** It stops the switch statement.
- ✓ **%d :** Format specifier for integer.
- ✓ **%f :** Format specifier for float.
- ✓ **%c :** Format specifier for character.
- ✓ **&& :** Logical AND.
- ✓ **|| :** Logical OR.

- if → one check
- if else → two choices
- else if → many conditions
- switch → fixed meny options

Activity 1 : Take input from keyboard , store values in variables , calculate average & display

```
  GNU nano 7.2                                              Lab2_A1.c *
#include <stdio.h>
int main()
{
  int mark1,mark2;
  float average;

  printf("Input the 1st mark:");
  scanf("%d",&mark1);

  printf("Input the 2nd mark:");
  scanf("%d",&mark2);

  average=(mark1+mark2)/2.0;
  printf("The average is:%f\n", average);

  return 0;
}
```

```
mathuppriya-nakuleswaran@mathuppriya-nakuleswaran-VirtualBox:~$ nano Lab2_A1
mathuppriya-nakuleswaran@mathuppriya-nakuleswaran-VirtualBox:~$ nano Lab2_A1.c
mathuppriya-nakuleswaran@mathuppriya-nakuleswaran-VirtualBox:~$ gcc -o Lab2_A1 Lab2_A1.c
mathuppriya-nakuleswaran@mathuppriya-nakuleswaran-VirtualBox:~$ ./Lab2_A1
Input the 1st mark:85
Input the 2nd mark:78
The average is:81.500000
mathuppriya-nakuleswaran@mathuppriya-nakuleswaran-VirtualBox:~$
```

Figure 2.1: Write in editor then Compile.

Explanation of the code : This program asks the user to enter two marks. It stores them in two integer variables and calculates the average by adding them and dividing by 2.0. The result is stored in a float variable because the average can contain decimal values. Finally, the program displays the average on the screen.

Line NO	Code	Explanation
1	#include <stdio.h>	This gives the program access to standard input and output functions like: ◆ printf() ◆ scanf().
2	int main()	This is where the program starts running.
3	int mark1, mark2;	These 2 variables store the marks entered by the user. They are integers because marks are usually whole numbers.

4	float average;	The average may contain decimal values, so we use float.
5	printf("Input the 1st mark: ");	This shows a message asking the user to type the first mark.
6	scanf("%d", &mark1);	This reads an integer from the keyboard and stores it in mark1.
7	printf("Input the 2nd mark: ")	This asks for the second mark.
8	scanf("%d", &mark2);	This stores the second mark in mark2.
9	average = (mark1 + mark2) / 2.0;	This adds the two marks and divides by 2.0. Using 2.0 is important because it forces decimal division.
10	printf("The average is: %f\n", average);	This prints the average as float value.
11	return 0;	This ends the program successfully.

- **Why is average declared as float :** Because the result may have decimal values.
- **&mark1 in scanf() :** scanf() needs the memory address of the variable to store the input value.
- **Why 2.0 and not 2 :** 2.0 makes the division decimal, so the average is more accurate.
- **%d :** format specifier for an integer.
- **%f :** format specifier for a floating-point value.
- **&mark1 :** The memory address where the value should be stored.(scanf() needs the memory address to store the input.)

Don't forget to when write

scanf("%d", mark1); **scanf("%d", &mark1);**

Activity 2 : if-else statement.

(Now, the program is not only calculating, but also deciding based on a condition.)

- Only one path happens.

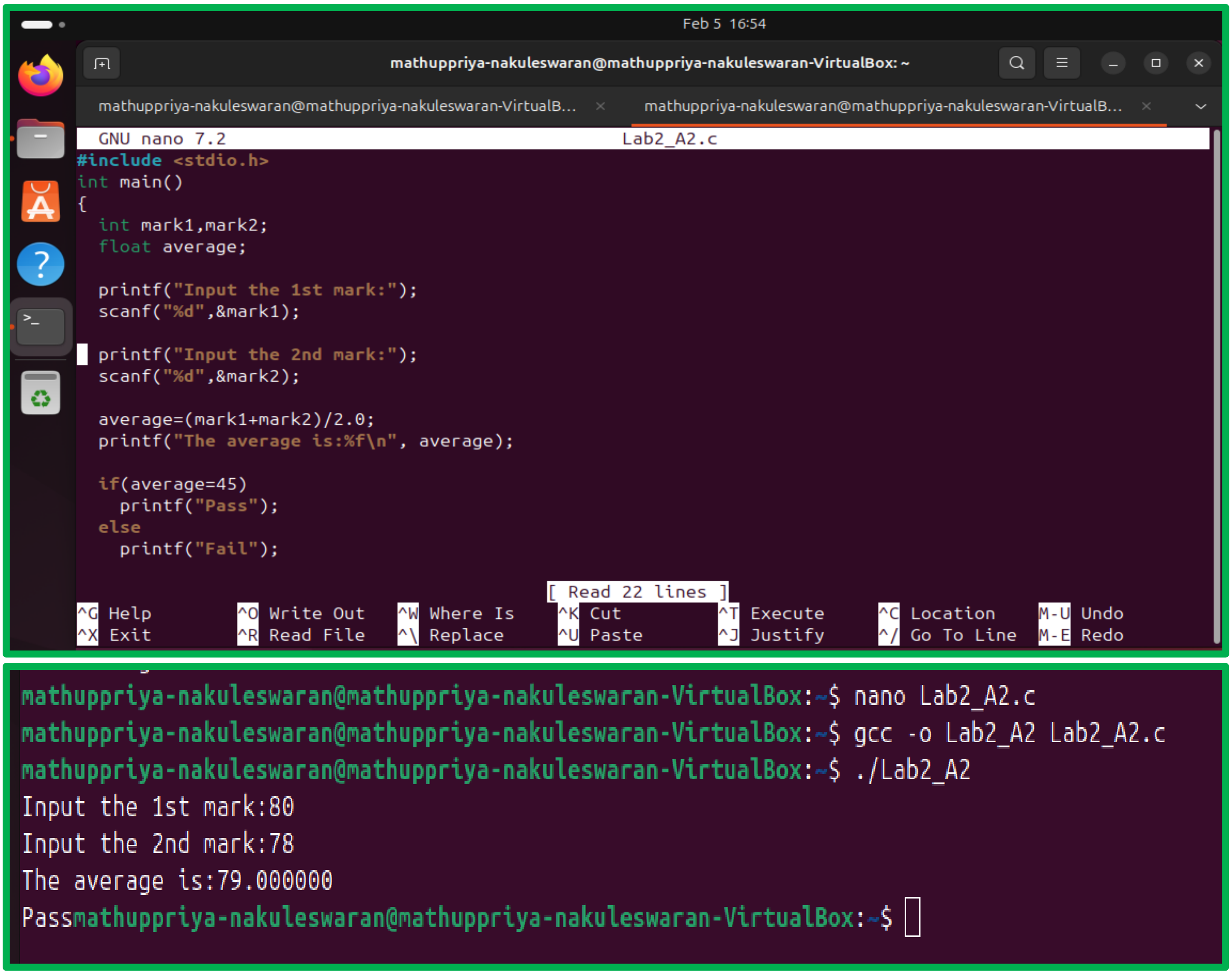
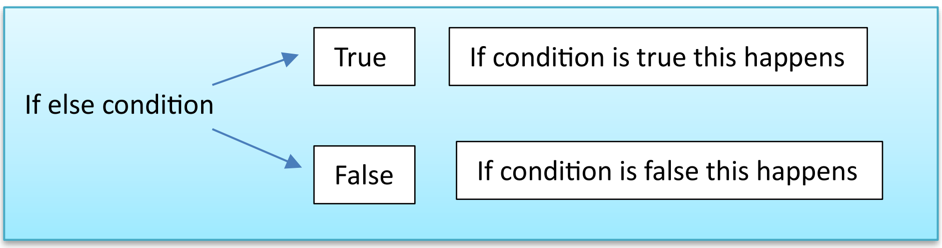


Figure 2.2: Write editor then Compile.

Explanation of the code : Ask 2 numbers from user then add both numbers ,then calculate average of the values. After calculating the average of two marks, it checks whether the average is greater than 45. If it is, the program displays “Pass”. Otherwise, it displays “Fail”.

Code	Explanation
if (average > 45)	This checks whether the average is greater than 45.
printf("Pass\n");	If the condition is true, print Pass.
Else	If the condition is false, go to the other option.
printf("Fail\n");	If average is not above 45, print Fail.



- **purpose of if-else :** It is used to choose between two paths based on a condition.
- if condition average > 45 : average = 45 : false, so it will print Fail.
- >= 45 : Then 45 will count as Pass.
- What type of value does a condition return : True or false

Activity 3 : To combine multiple conditions in one program.

char : only one letter is needed.

Why is there a space before %c in scanf(" %c", &gender) : To skip unwanted whitespace or newline characters. Don't forget the space before %c.

|| **:** Logical OR.

nested if : we first check age, then inside that check gender.

if (gender == 'M' || gender == 'm')

This accept both small letter or capital letter

%c -> char

%d -> int

%f -> float


```
#include <stdio.h>
int main()
{
char gender;
int age;

printf("Enter gender(M/F): ");
scanf("%c",&gender);

printf("Enter age:");
scanf("%d",&age);

if(age>=65){
  if(gender=='M'|| gender=='m')
   printf("Senior Male citizen");

   else if(gender=='F' || gender=='f')
    printf("Senior Female citizen");

    else
     printf("Invalide gender");
   } else{
     printf("Not a senior citizen");
   }
```

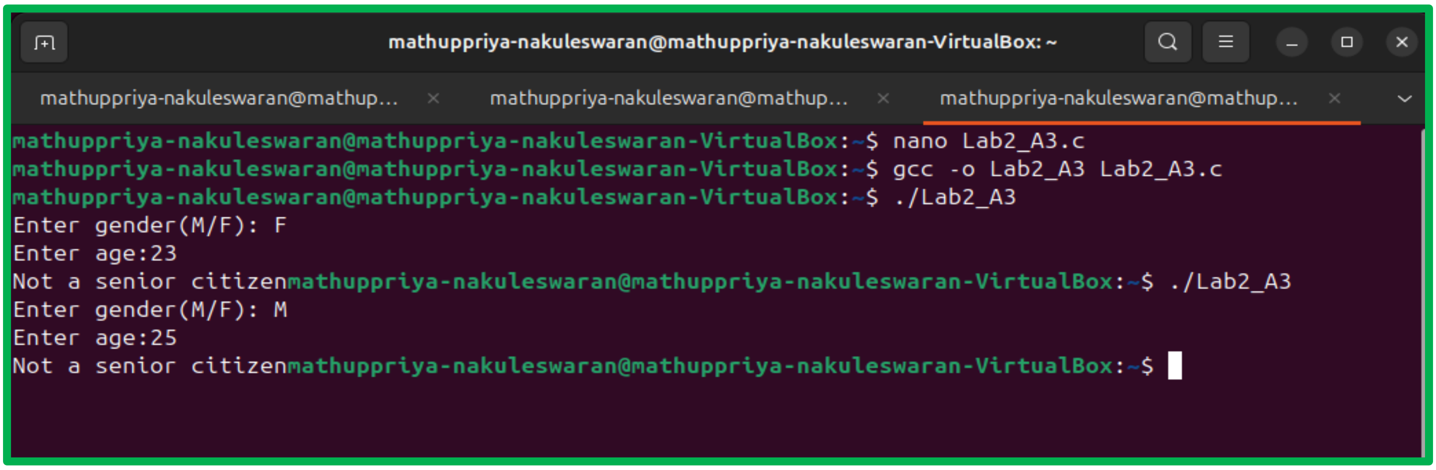


Figure 2.1: Write editor then Compile.

Explanation of the code : This program takes the gender and age of a person as input. It first ch whether the person is 65 years old or above. If the person is a senior citizen, the program then ch the gender and displays either "Senior Male citizen" or "Senior Female citizen". If the age is below 6 displays that the person is not a senior citizen.

Code	Explanation
char gender;	Stores one character like M or F.
scanf(" %c", &gender);	Reads one character from keyboard. The space before %c is important because it helps ignore leftover spaces or Enter key input.
if (age >= 65)	Checks whether the person is a senior citizen.
if (gender == 'M' || gender == 'm')	Checks gender is male, can be uppercase / lowercase.
else if (gender == 'F' || gender == 'f')	Checks whether gender is female.
else	Handles wrong input like X.

Activity 4 : if-else

- if-else for checking ranges.
- we have multiple ranges, so a chain of conditions is needed.
- **else if** : Because there are several different ranges to check.
- **&&** = Logical AND. Both conditions must be true.

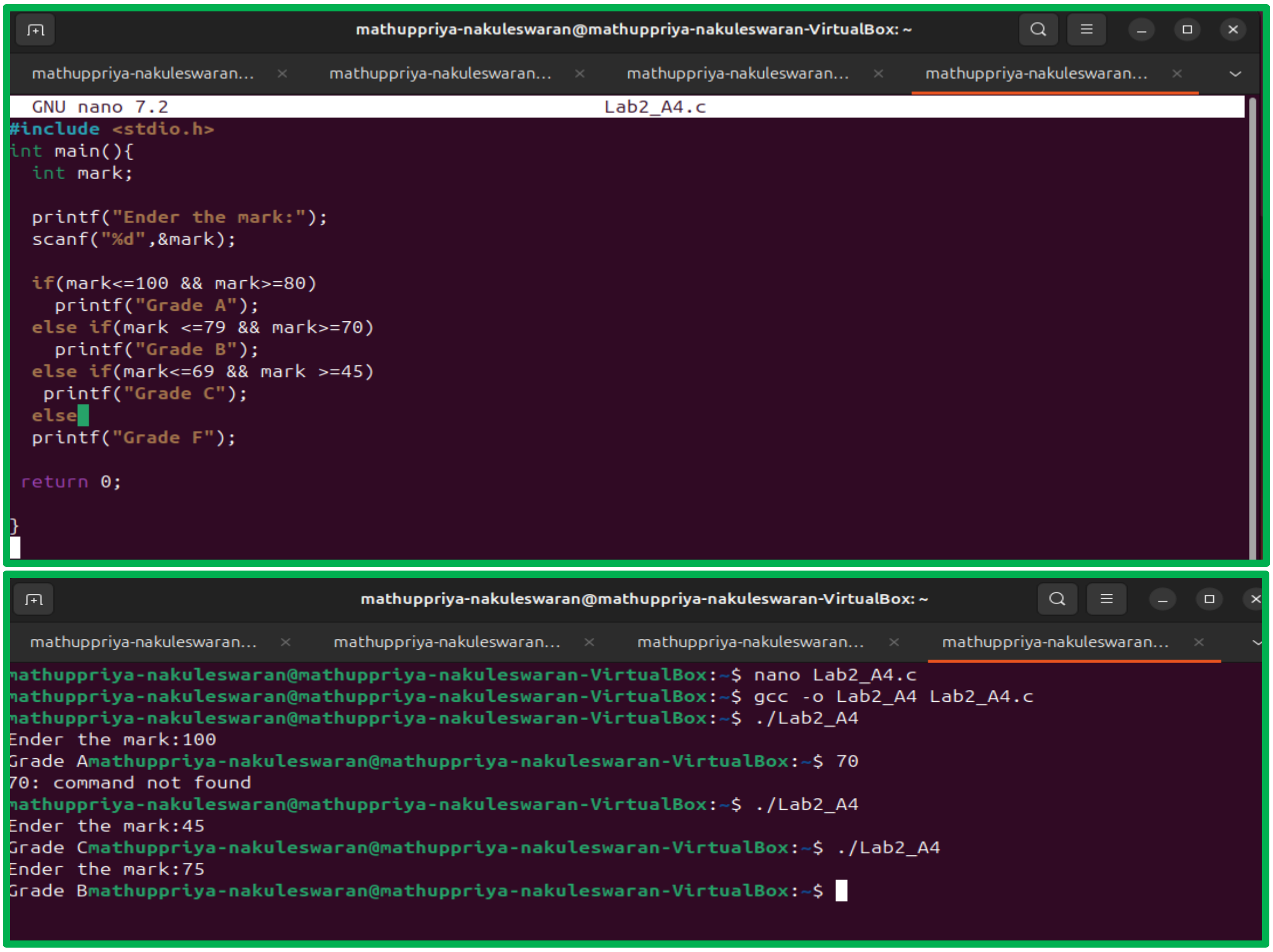


Figure 2.1: Write editor then Compile.

Explanation of the code : This program reads a module mark from the keyboard and checks which range the mark belongs to. If the mark is between 80 and 100, it displays grade A. If it is between 70 and 79, it displays B. If it is between 45 and 69, it displays C. If the mark is below 45, it displays F. If the entered value is outside the valid mark range, the program displays an invalid mark message.

Why should we validate the mark :Because marks should not be below 0 or above 100.

Why is order important in else if : Because the program checks from top to bottom and stops at the first true condition.

In math, you can write: 80 <= mark <= 100
But in C, you must write: mark >= 80 && mark <= 100

code	Explanation
if (mark >= 80 && mark <= 100)	If the mark is from 80 to 100, print A.
else if (mark >= 70 && mark <= 79)	If not A, then check whether it belongs to B range.
else if (mark >= 45 && mark <= 69)	Then check C range.
else if (mark >= 0 && mark < 45)	Then check fail range.
else	If the mark is negative or above 100, input is invalid.

Activity 5 : Switch-case

A **switch statement** is used to execute one block among many based on the value of an expression.

It also says that if-else is used for conditions and ranges, while switch-case is better when selecting from a fixed set of constant values.

When is switch-case better than if-else : When there are fixed constant choices like 1, 2, & 3.

Purpose of break : It stops execution after the correct case runs.

if break is missing : The program may continue into the next case. This is called fall-through.

default : It handles invalid or unmatched values.

case choice == 1:	wrong	switch (choice) { case 1:	right

```
mathuppriya-nakuleswaran@mathuppriya-nakuleswaran-VirtualBox: ~
mathuppriya-nakule...   mathuppriya-nakule...   mathuppriya-nakule...   mathuppriya-nakule...   mathuppriya-nakule...
mathuppriya-nakuleswaran@mathuppriya-nakuleswaran-VirtualBox:~$ nano Lab2_A5.c
mathuppriya-nakuleswaran@mathuppriya-nakuleswaran-VirtualBox:~$ gcc -o Lab2_A5 Lab2_A5.c
mathuppriya-nakuleswaran@mathuppriya-nakuleswaran-VirtualBox:~$ ./Lab2_A5
Menu
1.Hello
 2.Welcome
3.Goodbye
Enter your choice:1
Hellomathuppriya-nakuleswaran@mathuppriya-nakuleswaran-VirtualBox:~$ ./Lab2_A5
Menu
1.Hello
 2.Welcome
3.Goodbye
Enter your choice:4
Invalid choicemathuppriya-nakuleswaran@mathuppriya-nakuleswaran-VirtualBox:~$
```


```c
#include <stdio.h>
int main(){
 int choice;

 printf("Menu\n");
 printf("1.Hello\n ");
 printf("2.Welcome\n");
 printf("3.Goodbye\n");
 pirntf("Enter your choice:");
 scanf("%d",&choice);

 switch(choice){
   case 1:
     printf("Hello");
     break;
   case 2:
     printf("Welcome");
     break;
   case 3:
     printf("Goodbye");
     break;
   default:
     printf("Invalid choice");
   }
    return 0;
}
```

Figure 2.1: This program displays a menu with three options and asks the user to enter a number. It uses a switch-case statement to match the entered number with one of the menu options. If the user enters 1, it prints Hello. If the user enters 2, it prints Welcome. If the user enters 3, it prints Goodbye. For any other value, the program prints Invalid choice.

Code	Explanation
switch (choice)	The program checks the value of choice.
case 1:	If choice is 1, this block runs.
break;	Stops the switch. Without it, the next case may also run.
default:	Runs when no case matches.

Activity 6

- Normally, switch-case works best with exact values, not ranges.
- But ranges can still be handled cleverly by switching on : mark >= 80 && mark <= 100

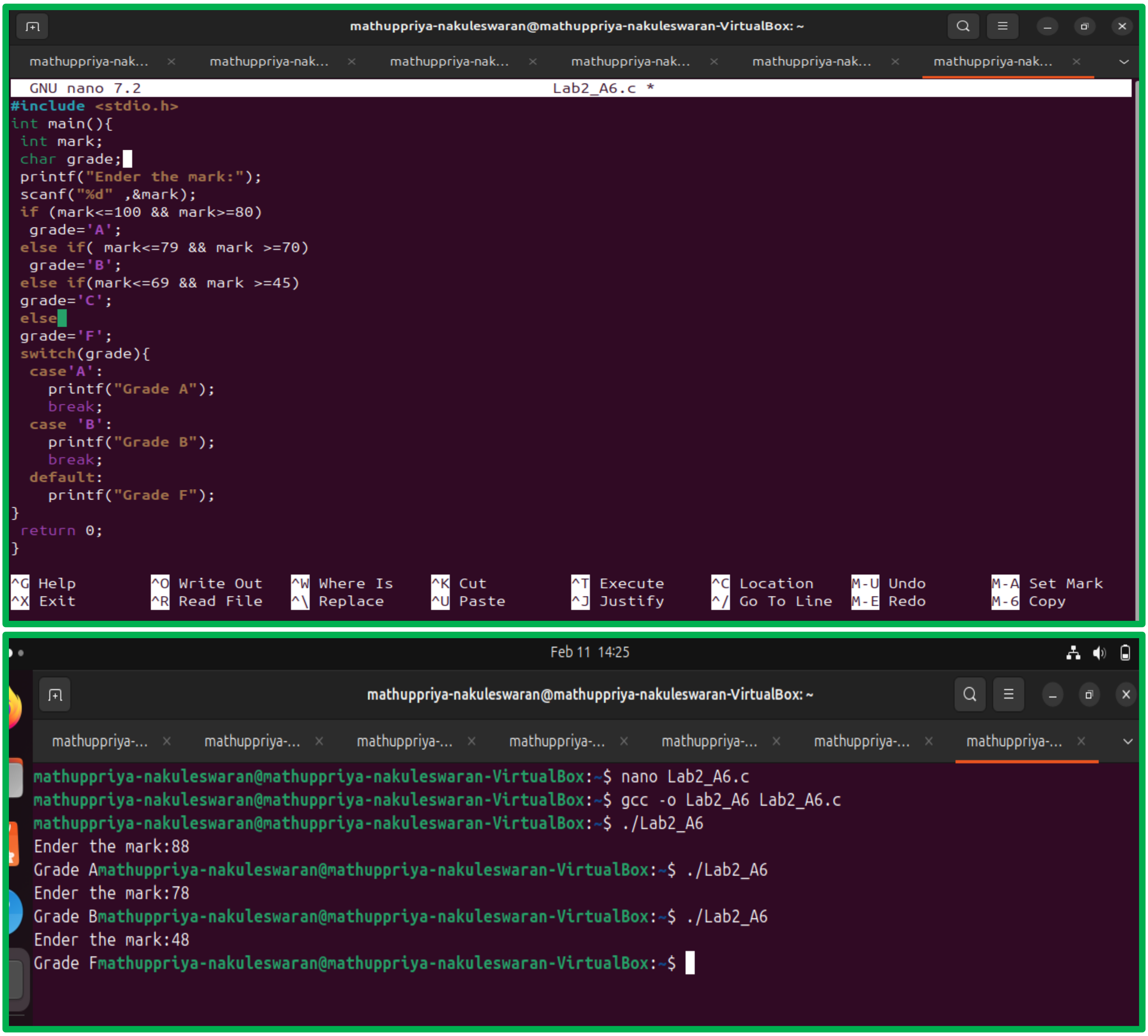


Figure 2.1: Write editor then Compile.

How to create and run the code

1. **open terminal**
2. **create a file** [nano activity1.c]
3. **compile** [gcc activity1.c -o activity1]
4. **run** [./activity1]

Common errors and fixes

1. gcc: command not found **->** Reason: GCC is not installed.

To Fix:

```
sudo apt update
sudo apt install gcc
```

2. expected ';' before ... **->** Reason: missing semicolon.

Fix: check the previous line.

3. wrong output for average **->** Reason: used 2 instead of 2.0, or used int for average.

Fix: use float average; and divide by 2.0.

4. gender input not working **->** Reason: %c reads leftover newline.

Fix: use:

```
scanf(" %c", &gender);
```

5. switch-case prints multiple messages **->** Reason: missing break.

Fix: add break; after each case.

6. invalid comparison syntax

Wrong: 80 <= mark <= 100

Correct: mark >= 80 && mark <= 100